```
run:
Nama : airil
NIM  : 13020240075
Jml MK: 3
Nilai MK-1: 90
Nilai MK-2: 95
Nilai MK-3: 100

=== RAPOR ===
Nama : airil
NIM  : 13020240075
Nilai:
 MK-1 : 90
 MK-2 : 95
 MK-3 : 100
Rata : 95.00
Grade : A
Lulus : true
BUILD SUCCESSFUL (total time: 1 minute 2 seconds)
```

1. Bagian Input (Data yang Dimasukkan)

Bagian awal menunjukkan proses di mana pengguna memasukkan data ke dalam sistem:

- Nama & NIM: Identitas mahasiswa atas nama airil dengan nomor induk 13020240075.
- Jml MK (3): Pengguna menentukan bahwa ada 3 mata kuliah yang akan dihitung nilainya.
- Nilai MK-1 s/d MK-3: Nilai yang dimasukkan secara berurutan adalah 90, 95, dan 100.

2. Bagian Laporan (RAPOR)

Setelah data diproses, program menampilkan ringkasan atau hasil kalkulasi:

- Daftar Nilai: Menampilkan kembali nilai-nilai yang sudah diinput agar bisa diverifikasi.
- Rata (95.00): Program menghitung nilai rata-rata dari ketiga mata kuliah tersebut.

$$90 + 95 + 100 = 3$$

$$= 95.00$$

- Grade (A): Berdasarkan nilai rata-rata 95, program memberikan predikat huruf A.
- Lulus (true): Status kelulusan dinyatakan true (benar/lulus), yang berarti nilai rata-rata airil memenuhi ambang batas minimal kelulusan yang diatur dalam kode program.

3. Status Sistem

- BUILD SUCCESSFUL: Menunjukkan bahwa kode program berjalan dengan sempurna tanpa ada kesalahan (*error*) teknis.
- Total time: Waktu yang dibutuhkan sistem untuk menjalankan proses tersebut dari awal hingga selesai (1 menit 2 detik).

Singkatnya: Ini adalah program Kalkulator Nilai Mahasiswa yang menerima input nama, NIM, dan deretan nilai, lalu secara otomatis menghitung rata-rata, menentukan grade, dan mengecek status kelulusan.

```
run:
Masukkan angka 1: 5
Masukkan angka 2: 3
Operator (+, -, *, /): +
Hasil: 5.00 + 3.00 = 8.00
Hitung lagi? (y/n): y

Masukkan angka 1: 10
Masukkan angka 2: 0
Operator (+, -, *, /): +
Hasil: 10.00 + 0.00 = 10.00
Hitung lagi? (y/n): n

Program selesai.
BUILD SUCCESSFUL (total time: 52 seconds)
```

1. Perhitungan Pertama (Iterasi 1)

Pada tahap ini, pengguna melakukan operasi penjumlahan sederhana:

- Input Angka: Pengguna memasukkan angka 5 dan 3.
- Operator: Pengguna memilih operator tambah (+).
- Hasil: Program menampilkan hasil dalam format desimal, yaitu 5.00 + 3.00 = 8.00.
- Konfirmasi: Muncul pertanyaan "Hitung lagi? (y/n)". Pengguna menjawab 'y' (yes), sehingga program mengulang proses dari awal.

2. Perhitungan Kedua (Iterasi 2)

Program kembali meminta input untuk perhitungan baru:

- Input Angka: Pengguna memasukkan angka 10 dan 0.
- Operator: Pengguna kembali memilih operator tambah (+).
- Hasil: Program menampilkan 10.00 + 0.00 = 10.00.
- Konfirmasi: Saat ditanya "Hitung lagi? (y/n)", pengguna menjawab 'n' (no).

3. Akhir Program

- Program selesai: Karena pengguna memilih 'n', program keluar dari loop dan menampilkan pesan penutup.
- BUILD SUCCESSFUL: Menunjukkan kompilasi dan eksekusi berjalan lancar tanpa kendala teknis dalam waktu 52 detik.

Kesimpulan: Program ini menggunakan logika Do-While atau While Loop, di mana program akan terus mengeksekusi blok kode perhitungan selama kondisi variabel konfirmasi bernilai 'y'.

```
run:
Masukkan total detik: 13020240075
Total detik: 13020240075
================
Waktu: 05:21:15
================
Total Detik    : 13020240075
detikSaat ini  : 15
menitSaat ini  : 21
jamSaat ini    : 5
Total Menit    : 217004001
Total Jam      : 3616733

--- Contoh lain ---
Input : 3661
Output: 01:01:01

Input : 86400
Output: 00:00:00 (tepat 1 hari)
BUILD SUCCESSFUL (total time: 1 minute 16 seconds)
```

1. Proses Konversi Utama

Program menerima input berupa total detik yang sangat besar dan memecahnya menjadi komponen waktu:

- Waktu: 05:21:15: Ini adalah format ringkas (Jam:Menit:Detik).
- detikSaat ini (15): Hasil dari sisa bagi total detik dengan 60 (modulus).
- menitSaat ini (21): Sisa menit setelah dikonversi ke jam.
- jamSaat ini (5): Sisa jam dalam format 24 jam.
- Total Menit & Total Jam: Merupakan konversi langsung dari total detik ke satuan menit dan jam secara keseluruhan tanpa memperhitungkan sisa (pembulatan ke bawah).

2. Bagian "Contoh Lain"

Program ini juga memberikan demonstrasi logika konversi untuk angka yang lebih sederhana agar mudah dipahami:

- Input 3661: Menghasilkan 01:01:01.
    - 3600 detik = 1 jam.
    - 60 detik = 1 menit.
    - 1 detik sisa.
- Input 86400: Menghasilkan 00:00:00 (tepat 1 hari). Ini karena 86.400 detik adalah jumlah detik dalam 24 jam penuh, sehingga pada format jam digital 24 jam, angkanya kembali ke nol.

3. Logika Pemrograman

Berdasarkan output tersebut, program ini kemungkinan besar menggunakan operator aritmatika berikut:

1. Modulus (%): Untuk mendapatkan sisa detik dan menit.
2. Pembagian (/): Untuk mendapatkan total jam dan menit.

Kesimpulan: Program ini bertujuan untuk mendemonstrasikan bagaimana sebuah angka bulat (integer) yang merepresentasikan durasi dalam detik dapat diformat menjadi penunjuk waktu yang manusiawi (human-readable).

```
run:

--- Katalog Toko Buah ---
1. Apel   Rp 15000 (stok:10 kg)
2. Jeruk   Rp 12000 (stok:8 kg)
3. Mangga   Rp 20000 (stok:5 kg)

Pilih (0=stop): 1
Jumlah (kg): 5
OK +Rp75000

--- Katalog Toko Buah ---
1. Apel   Rp 15000 (stok:5 kg)
2. Jeruk   Rp 12000 (stok:8 kg)
3. Mangga   Rp 20000 (stok:5 kg)

Pilih (0=stop): 2
Jumlah (kg): 1
OK +Rp12000

--- Katalog Toko Buah ---
1. Apel   Rp 15000 (stok:5 kg)
2. Jeruk   Rp 12000 (stok:7 kg)
3. Mangga   Rp 20000 (stok:5 kg)

Pilih (0=stop): 3
Jumlah (kg): 2
OK +Rp40000

--- Katalog Toko Buah ---
1. Apel   Rp 15000 (stok:5 kg)
2. Jeruk   Rp 12000 (stok:7 kg)
3. Mangga   Rp 20000 (stok:3 kg)

Pilih (0=stop): 0
Total: Rp127000
BUILD SUCCESSFUL (total time: 49 seconds)
```

1. Daftar Katalog dan Stok Awal

Program menampilkan daftar buah beserta harga per kilogram dan jumlah stok yang tersedia:

- Apel: Rp 15.000 (10 kg)
- Jeruk: Rp 12.000 (8 kg)
- Mangga: Rp 20.000 (5 kg)

2. Proses Transaksi (Iterasi Belanja)

Pengguna melakukan tiga kali pembelian sebelum akhirnya berhenti:

- Pembelian 1 (Apel): Pengguna membeli 5 kg Apel.
    - Sistem menghitung: $5 \times 15.000 = 75.000$.
    - Penting: Stok Apel berkurang dari 10 kg menjadi 5 kg pada tampilan berikutnya.
- Pembelian 2 (Jeruk): Pengguna membeli 1 kg Jeruk.
    - Sistem menghitung: $1 \times 12.000 = 12.000$.
    - Stok Jeruk berkurang dari 8 kg menjadi 7 kg.
- Pembelian 3 (Mangga): Pengguna membeli 2 kg Mangga.
    - Sistem menghitung: $2 \times 20.000 = 40.000$.
    - Stok Mangga berkurang dari 5 kg menjadi 3 kg.

3. Total Akhir dan Penghentian

- Pilih (0=stop): Pengguna memasukkan angka 0, yang merupakan instruksi untuk keluar dari perulangan (*break loop*).
- Total: Rp 127.000: Ini adalah hasil penjumlahan dari seluruh transaksi tadi:

  75.000 + 12.000 + 40.000 = 127.000

Konsep Pemrograman yang Digunakan

1. Looping (Perulangan): Menggunakan while atau do-while agar pembeli bisa menambah item terus-menerus sampai menekan angka 0.
2. Update Variable: Variabel stok buah diperbarui nilainya setiap kali transaksi berhasil (stok = stok - jumlahBeli).
3. Accumulator: Variabel total belanja terus bertambah (total = total + hargaBayar).
4. Array/Object: Data buah (nama, harga, stok) kemungkinan disimpan dalam struktur data seperti Array atau List agar mudah dikelola.